

Week7

Task1:

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    char command[200];
    char filename[100];
    int saved_stdin, fd;
    printf("Enter command with input redirection:\n");
    printf("Example: cat < input.txt\n");
    printf("Command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';
    char *redirect = strstr(command, "<");
    if (redirect == NULL)
    {
        printf("No input redirection found.\n");
        return 0;
    }
    *redirect = '\0';
    char *cmd = command;
    char *file = redirect + 1;
    while (*file == ' ')
        file++;
    strcpy(filename, file);
    printf("\nCommand: %s\n", cmd);
    printf("Input file: %s\n", filename);
    fd = open(filename, O_RDONLY);
    if (fd == -1)
    {
        perror("Error opening input file");
        return 1;
    }
    saved_stdin = dup(STDIN_FILENO);
    if (saved_stdin == -1)
    {
        perror("dup");
        close(fd);
    }
}
```

```

    {
        perror("dup");
        close(fd);
        return 1;
    }
    if (dup2(fd, STDIN_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        close(saved_stdin);
        return 1;
    }
    close(fd);
    printf("\nReading input from file...\n");
    char buffer[500];
    while (fgets(buffer, sizeof(buffer), stdin) != NULL)
    {
        printf("%s", buffer);
    }
    if (dup2(saved_stdin, STDIN_FILENO) == -1)
    {
        perror("Failed to restore stdin");
        close(saved_stdin);
        return 1;
    }
    close(saved_stdin);
    printf("\n\nstdin restored successfully.\n");
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./redirect
Enter command with input redirection:
Example: cat < input.txt
Command: cat < missing.txt

Command: cat
Input file: missing.txt
Error opening input file: No such file or directory
root@DESKTOP-VMN932K:~# _

```

Task2:

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    char command[200];
    char filename[100];
    int saved_stdout, fd;
    printf("Enter command with output redirection:\n");
    printf("Example: ls > output.txt\n");
    printf("Command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';
    char *redirect = strstr(command, ">");
    if (redirect == NULL)
    {
        printf("No output redirection found.\n");
        return 0;
    }
    *redirect = '\0';
    char *cmd = command;
    char *file = redirect + 1;
    while (*file == ' ')
        file++;
    strcpy(filename, file);
    printf("\nCommand: %s\n", cmd);
    printf("Output file: %s\n", filename);
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        perror("Error opening output file");
        return 1;
    }
    saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout == -1)
    {
        perror("dup");
        close(fd);

```

```

        perror("dup");
        close(fd);
        return 1;
    }
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        close(saved_stdout);
        return 1;
    }
    close(fd);
    printf("This output is redirected to the file.\n");
    printf("Output redirection is working successfully.\n");
    if (dup2(saved_stdout, STDOUT_FILENO) == -1)
    {
        perror("Failed to restore stdout");
        close(saved_stdout);
        return 1;
    }
    close(saved_stdout);
    printf("\nstdout restored successfully.\n");
    printf("Check '%s' to verify the output data.\n", filename);
    return 0;
}

```

Output:

```
Enter command with output redirection:
Example: ls > output.txt
Command: ls > output.txt

Command: ls
Output file: output.txt

stdout restored successfully.
Check 'output.txt' to verify the output data.
root@DESKTOP-VMN932K:~#
```

Task3:

To Parse Append Operator, Open Files in Append Mode,
Redirect Streams, Preserve Existing Data, Validate Appends,
Test Scenarios

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    char command[200];
    char filename[100];
    int saved_stdout, fd;
    printf("Enter command with append redirection:\n");
    printf("Example: echo Hello >> output.txt\n");
    printf("Command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';
    char *redirect = strstr(command, ">>");
    if (redirect == NULL)
    {
        printf("No append redirection found.\n");
        return 0;
    }
    *redirect = '\0';
    char *cmd = command;
    char *file = redirect + 2;
    while (*file == ' ')
        file++;
    strcpy(filename, file);
    printf("\nCommand: %s\n", cmd);
    printf("Output file: %s\n", filename);
    fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd == -1)
    {
        perror("Error opening output file");
        return 1;
    }
    saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout == -1)
    {
        perror("dup");
        close(fd);
    }
```

```

        perror("dup");
        close(fd);
        return 1;
    }
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        close(saved_stdout);
        return 1;
    }
    close(fd);
    printf("New data appended to the file.\n");
    printf("Existing data is preserved.\n");
    if (dup2(saved_stdout, STDOUT_FILENO) == -1)
    {
        perror("Failed to restore stdout");
        close(saved_stdout);
        return 1;
    }
    close(saved_stdout);
    printf("\nstdout restored successfully.\n");
    printf("Append operation completed.\n");
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./append2
Enter command with append redirection:
Example: echo Hello >> output.txt
Command: echo Hello >> output.txt

Command: echo Hello
Output file: output.txt

stdout restored successfully.
Append operation completed.
root@DESKTOP-VMN932K:~# █

```

Task4:

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    char command[200];
    char filename[100];
    int saved_stderr, fd;
    printf("Enter command with stderr redirection:\n");
    printf("Example: ls wrongfile 2> error.txt\n");
    printf("Command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';
    char *redirect = strstr(command, "2>");
    if (redirect == NULL)
    {
        printf("No stderr redirection found.\n");
        return 0;
    }
    *redirect = '\0';
    char *cmd = command;
    char *file = redirect + 2;
    while (*file == ' ')
        file++;
    strcpy(filename, file);
    printf("\nCommand: %s\n", cmd);
    printf("Error file: %s\n", filename);
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        perror("Error opening error file");
        return 1;
    }
    saved_stderr = dup(STDERR_FILENO);
    if (saved_stderr == -1)
    {
        perror("dup");
        close(fd);
    }
```

```

    if (saved_stderr == -1)
    {
        perror("dup");
        close(fd);
        return 1;
    }
    if (dup2(fd, STDERR_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        close(saved_stderr);
        return 1;
    }
    close(fd);
    fprintf(stderr, "This is an error message.\n");
    fprintf(stderr, "Error output has been redirected successfully.\n");
    if (dup2(saved_stderr, STDERR_FILENO) == -1)
    {
        perror("Failed to restore stderr");
        close(saved_stderr);
        return 1;
    }
    close(saved_stderr);
    printf("\nstderr restored successfully.\n");
    printf("Check '%s' to verify the error output.\n", filename);
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./parse2
Enter command with stderr redirection:
Example: ls wrongfile 2> error.txt
Command: ls wrongfile 2> error.txt

Command: ls wrongfile
Error file: error.txt

stderr restored successfully.
Check 'error.txt' to verify the error output
root@DESKTOP-VMN932K:~# █

```